DAC - 203: ARTIFICIAL INTELLIGENCE | FINAL ACADEMIC RESEARCH THESIS

Adaptive Restart and Nogood Management for Constraint Satisfaction Solvers

A Quantitative Risk & Finance-Inspired Search Control Architecture in Glucose 4.2.1

V3 DRAWDOWN
DUAL EWMA & HIGH-WATER MARK
Dev Suite Winner (PAR-2 2.5381s)

ADAPTIVE-FIXED
TRI-CRITERIA CLAUSE SCORING
Holdout Winner (PAR-2 0.1107s)

100% VERIFIED
WINDOWS 64-BIT TESTBED
3,000+ Runs (Zero Errors)

👤 **Presented by:**
- **Manav Garg (25125026)**
- **Kugashiya Bhavy (25125024)**

🎓 **Faculty Advisor: Prof. Manoj Kumar**
🏛️ Department of Computer Science & Engineering

🗓️ November 2026 | Final Defense Presentation

Research Overview & Quantitative Architecture Dashboard

High-level problem, quantitative methodology, and rigorously verified empirical findings

THE CORE CHALLENGE

❌ **Search Stagnation:**
Classical CDCL solvers suffer from heavy-tailed runtimes when trapped in futile subtrees.

❌ **Rigid Static Policies:**
Luby restart sequences operate blind to real-time search volatility.

❌ **Clause Database Bloat:**
Unchecked nogood retention degrades 2WL unit propagation speed.

❓ **Core Question:**
Can financial risk metrics dynamically optimize search control?

QUANTITATIVE METHODS

📈 **V3 Drawdown Model:**
Tracks conflict return rt via dual EWMAs (fast=0.08, slow=0.01) and decaying HWM.

🛡️ **Adaptive-Fixed Retention:**
Continuous scoring: R(c) = 0.50 LBD + 0.40 Act + 0.10 Size with decoupled Glucose cadence.

⚠️ **Quantitative Risk Engine:**
Momentum (Sharpe < -0.35) and tail shock (VaR95) with 1500-conflict gating.

🎰 **Portfolio-MAB:**
Two-armed bandit for online restart policy switching.

VERIFIED RESULTS

🏆 **Development Suite Winner:**
V3 Drawdown + AdaptFix won the 36-benchmark development suite (PAR-2 2.5381s vs 2.7463s baseline; -15.5% conflicts).

🏆 **Holdout Suite Winner:**
Glucose + AdaptFix won the 48-benchmark holdout suite (PAR-2 0.1107s; 144/144 solved).

🎯 **Hanoi-5 Planning Speedup:**
V3 achieved 80.0% speedup over No-Heuristic control with surgical 5 restarts (3.06s vs 15.28s).

ENGINEERING & AUDIT

⚙️ **Native Windows 64-bit:**
MinGW-w64 GCC 14.2.0 build with native Win32 performance counters.

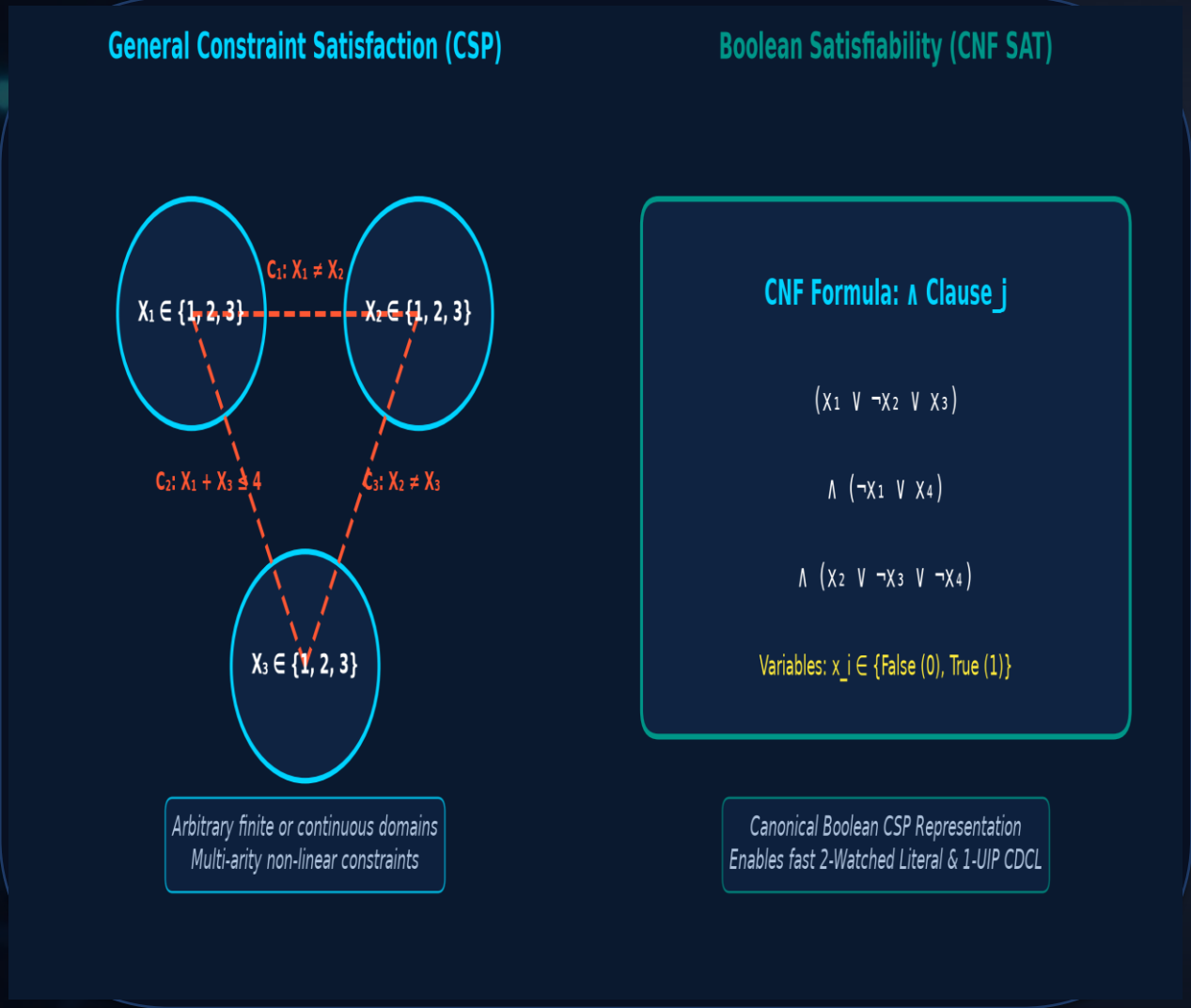
✅ **100% Model Verification:**
Every satisfying assignment across 3,000+ runs verified against raw CNF clauses.

🔒 **Cryptographic Manifests:**
Immutable SHA-256 benchmark hashes and frozen executables.

🛡️ **Scientific Integrity:**
Negative results on random SAT disclosed and stabilized.

Constraint Satisfaction & Boolean Satisfiability

Combinatorial complexity, CNF representation, and industrial relevance



GENERAL CONSTRAINT SATISFACTION (CSP)

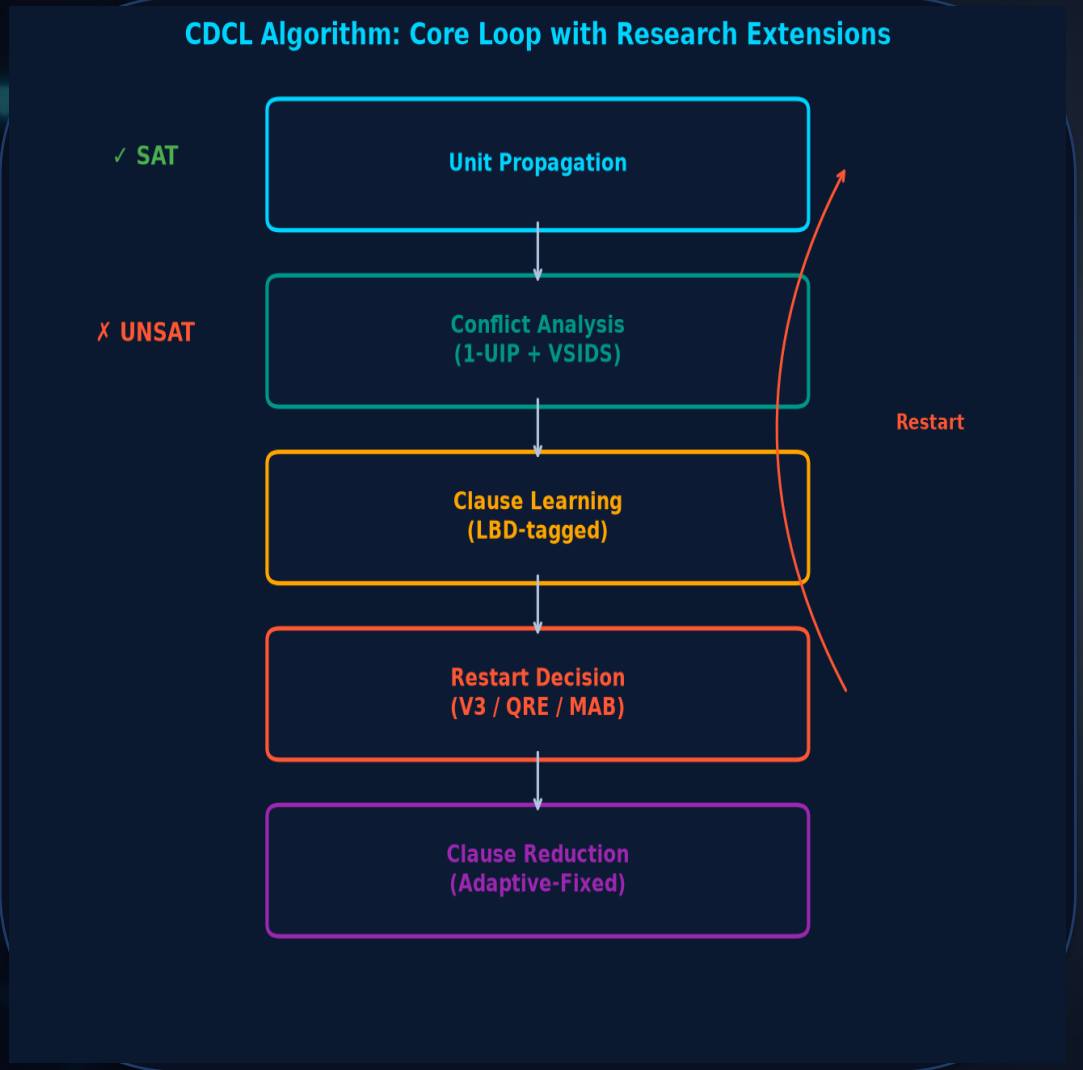
- Formal Definition: Tuple <X, D, C> where X is variables, D is domain, C is constraints.
- Combinatorial Complexity: Finding a consistent assignment is NP-complete in general.
- **Ubiquitous Real-World Applications:**
  Resource allocation, job-shop scheduling, vehicle routing, frequency assignment, and circuit layout.

BOOLEAN SAT: THE CANONICAL CORE

- Canonical Representation: Discrete CSPs encode into CNF SAT via Direct or Logarithmic encodings.
- Cook-Levin Theorem (1971): Established Boolean SAT as foundational NP-complete problem.
- **Industrial Significance:**
  Hardware formal verification (Intel, AMD), software model checking, cryptographic analysis, and automated AI planning.
- **The Scalability Challenge: Modern formulas contain millions of variables; naive backtracking fails completely.**

Inside Modern Conflict-Driven Clause Learning (CDCL)

Step-by-step execution cycle of modern industrial SAT solvers



01 **VSIDS Variable Branching**
Decays variable conflict activity by 0.95, focusing search on variables in active contention.

02 **Two-Watched Literals (2WL)**
Inspects clauses only when a watched literal is false, enabling tens of millions of propagations/s.

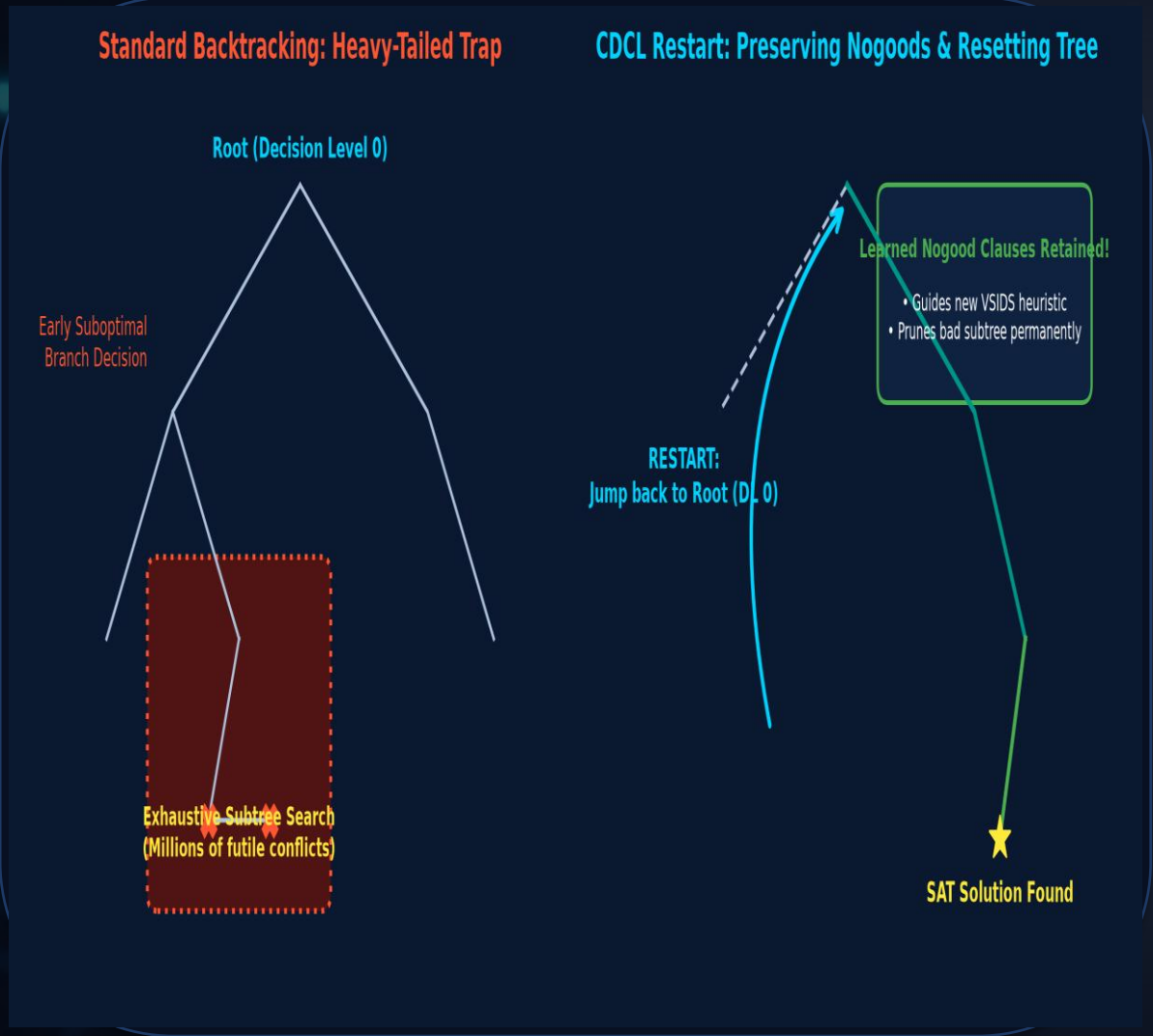
03 **1-UIP Conflict Analysis**
Traverses implication graph backwards to First Unique Implication Point to derive asserting nogood.

04 **Non-Chronological Backjumping**
Jumps directly to the asserting level of learned clause, permanently pruning unsatisfiable subtrees.

05 **Search Control: Restarts & Reduction**
Periodically cancels trail to escape local traps; regularly purges low-quality learned clauses.

The Dual Search-Control Dilemmas: Restarts & Clause Bloat

Why static restart heuristics and unchecked nogood accumulation degrade solving efficiency



DILEMMA I: THE RESTART DILEMMA

Heavy-Tailed Traps (Gomes et al.):

Backtrack search runtimes follow heavy-tailed Pareto distributions. Early poor decisions trap solvers in massive futile subtrees.

⚠️ Premature Restart Risk: Erases partial assignment trails before deep implications are uncovered, causing search thrashing.

⏳ Delayed Restart Risk: Static Luby sequences operate blind to real-time search state, wasting millions of conflicts in barren regions.

DILEMMA II: CLAUSE DATABASE BLOAT

Unbounded Clause Growth:

Learning one clause per conflict rapidly generates hundreds of thousands of learned nogoods, exhausting working memory.

Propagation Latency Degradation:

Excessive clauses bloat Two-Watched Literal lists, slowing BCP throughput by up to 300% on complex industrial formulas.

🗑️ Information Loss Tradeoff: Deleting too aggressively purges critical lemmas, forcing repeated rediscovery of conflict subtrees.

Four Foundational Pillars in Combinatorial Search

Seminal papers informing our adaptive restart and clause retention framework

Audemard & Simon (IJCAI 2009 / JSAT 2012)

Glucose: Predicting Learned Clause Quality Using LBD

- **Introduced Literal Block Distance (LBD): Number of distinct decision levels in a clause.**
- Core Insight: Clauses with LBD <= 2 ('Glue Clauses') are exceptionally powerful and must never be deleted.
- Dynamic Restarts: Luby sequence with moving average LBD comparisons to trigger rapid clearing.

Biere & Fröhlich (SAT 2018)

CaDiCaL: Chronological Backtracking & Inprocessing

- **Demonstrated that non-chronological backjumping can be augmented with chronological steps.**
- Highlighted the vital role of interleaved inprocessing and bounded variable elimination.
- Emphasized disciplined learned-clause garbage collection cadences.

Liang, Cherif, & Li (CP 2016 / 2026)

MAB Learning at the Right Time Scale for SAT

- **Applied Multi-Armed Bandits to select variable branching heuristics (CHB / LRB).**
- Key Lesson: Reward signals must match the temporal scale of search decisions.
- Showed non-stationary bandit algorithms track solver search mode shifts.

Tolpin & Shimony (2011) / Koriche et al. (2022)

Value of Information (VOI) and Adaptive Tournaments

- **Formally modeled the computational cost-benefit tradeoff of heuristic evaluations.**
- Adaptive Tournaments (AST): Eliminates suboptimal search policies early via short runs.
- Demonstrated online meta-heuristics can outperform statically tuned solvers.

Research Objectives & Technical Hypotheses

Three core questions driving our quantitative search control architecture



❓ **Research Question 1 (Adaptive Restarting): Can finance-inspired drawdown metrics (dual EWMA, High-Water Mark) detect search stagnation better than static Luby sequences?**

💡 Hypothesis: Monitoring propagation returns and drawdown collapse (Dt >= 0.75 for 30 consecutive conflicts) will prune futile subtrees while preserving productive momentum.

❓ **Research Question 2 (Adaptive Clause Retention): Can composite multi-attribute clause scoring (LBD + activity + size) resolve database bloat without causing legacy reduction deferral bugs?**

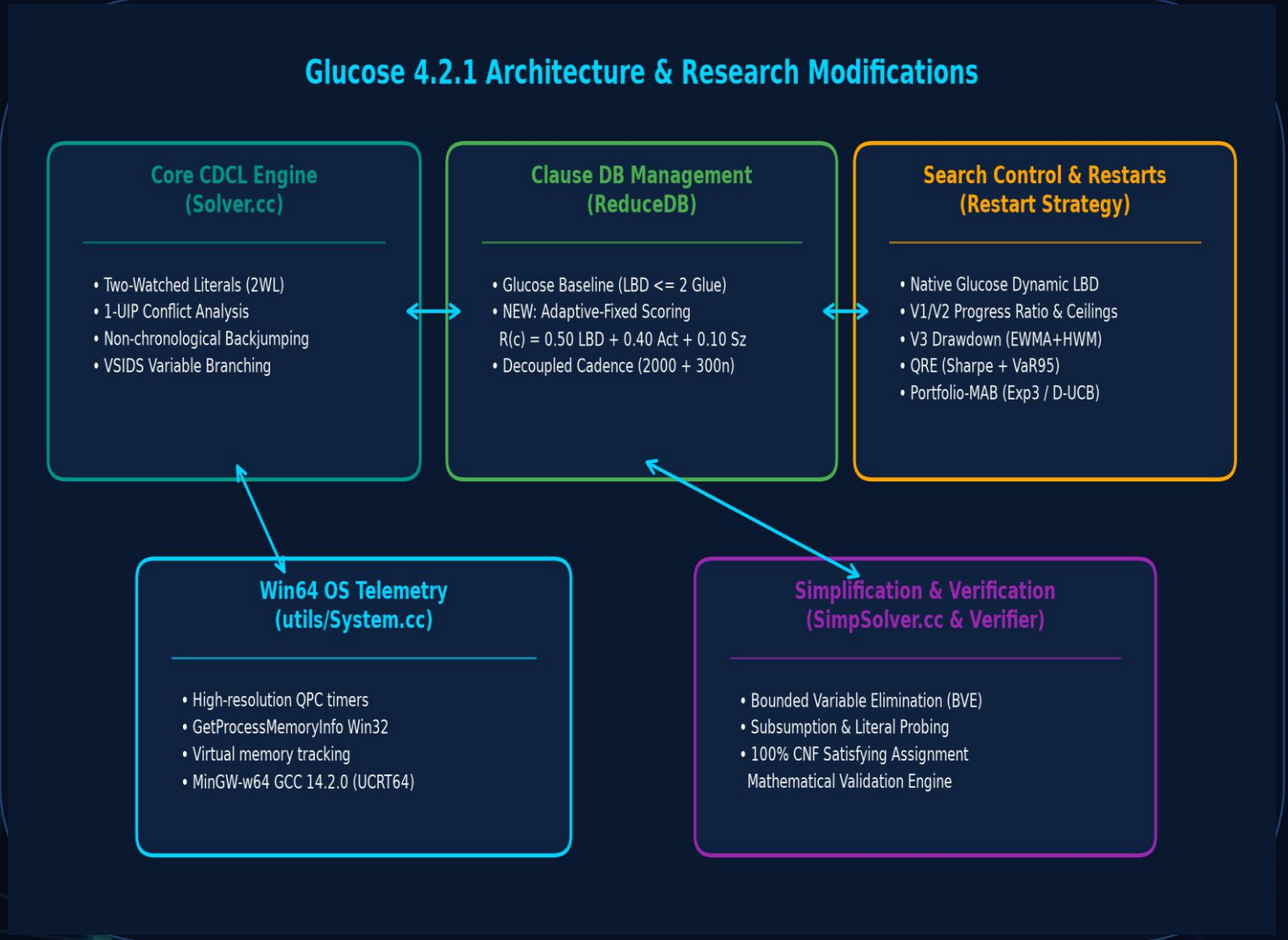
💡 Hypothesis: A decoupled Glucose cadence (2000+300n) coupled with R(c) = 0.50 LBD + 0.40 Act + 0.10 Sz will maintain lean memory and cut total conflicts.

❓ **Research Question 3 (Online Policy Portfolio): Can a non-stationary Multi-Armed Bandit (Exp3 / D-UCB) adaptively switch between exploration and bounded descent during search?**

💡 Hypothesis: Online learning on conflict-normalized rewards will dynamically match the optimal restart policy to instance difficulty.

Glucose 4.2.1 Implementation & Windows 64-bit Port

High-performance native Windows 64-bit testbed built under MinGW-w64 GCC 14.2.0



SOURCE MODIFICATIONS & ARTIFACTS

📁 **core/Solver.h / Solver.cc:**
~4,800 lines of modified C++. Implements all 9 restart strategies, 3 clause reduction algorithms, QRE telemetry, and MAB bandit controllers.

📁 **simp/SimpSolver.cc & Main.cc:**
Bounded variable elimination, literal probing, subsumption, and complete CLI option parsing.

📁 **utils/System.cc (Win32 Port):**
Native Windows process telemetry using QueryPerformanceCounter and GetProcessMemoryInfo.

⚙️ **Compiler Environment:**
MinGW-w64 GCC 14.2.0 (MSYS2 UCRT64). Flags: -O3 -std=c++11 -DNDEBUG -lz -lpsapi.

📦 **Cryptographically Frozen Binaries:**
- glucose-simp.exe (SHA-256 Step 35)
- glucose-qre-gated.exe (Step 40)
- glucose-portfolio-mab.exe (Step 44)

Reference Algorithms in Native Glucose 4.2.1

The reference components against which all research modifications are evaluated

1. VSIDS Variable Branching

- Variable State Independent Decaying Sum tracks conflict involvement.
- Decay Mechanism: Variable activity bumped upon conflict, multiplied by var_decay = 0.95.
- **Phase Saving: Reassigns variable to its previous polarity upon restart.**

2. Two-Watched Literals (2WL)

- Watches exactly 2 unassigned literals per clause in memory.
- Zero Overhead on False: Literals assigned false do not require list updates unless watched.
- **Enables tens of millions of propagations per second in C++.**

3. Glucose Dynamic LBD Restarts

- Computes Literal Block Distance (LBD) for every learned conflict clause.
- Moving Average Comparison: Fast LBD queue (size 50) vs slow cumulative average.
- **Trigger: If fast_avg * 0.80 > slow_avg, rapid restart is triggered to clear shallow trail.**

4. Baseline Clause Reduction (ReduceDB)

- Triggered at fixed intervals: firstReduceDB = 2000, incReduceDB = 300.
- Strict Protection: Clauses with LBD <= 2 (Glue clauses) are permanently preserved.
- **Sorting: Non-glue clauses sorted solely by VSIDS activity; bottom 50% purged.**

Adaptive Restart V1 & V2: Signal-Based Search Control

Initial implementations targeting propagation efficiency and anti-starvation ceilings

ADAPTIVE RESTART V1: PROGRESS RATIO

📊 **Core Efficiency Score (SW):**
Monitors unit propagation yield per decision level in a rolling circular buffer (ADAPT_WIN_CAPACITY = 50 conflicts):

🧮 **$S_W = \mu_{P,W} / (\sigma_{L,W} + 1.0)$**
where $\mu_{P,W}$ is mean propagations per conflict and $\sigma_{L,W}$ is level standard deviation.

🌐 **Global Baseline (S0):**
Cumulative baseline tracked via Welford's online variance algorithm from search start.

⚡ **Trigger Condition: $S_W < 0.60 * S_0$ (after 100 warmup and 50 min interval conflicts).**

⚠️ **V1 Limitation (Starvation Trap):**
On certain structured formulas, SW remained permanently above threshold, allowing the solver to run for 50,000+ conflicts without restarting!

ADAPTIVE RESTART V2: ANTI-STARVATION

🛡️ **V2 Anti-Starvation Ceiling:**
Forces an unconditional restart if search continues too long without progress:

🧮 **adapt_conflicts_since_restart >= 2500**
Guarantees the solver never remains trapped in a deep unsatisfiable subtree indefinitely.

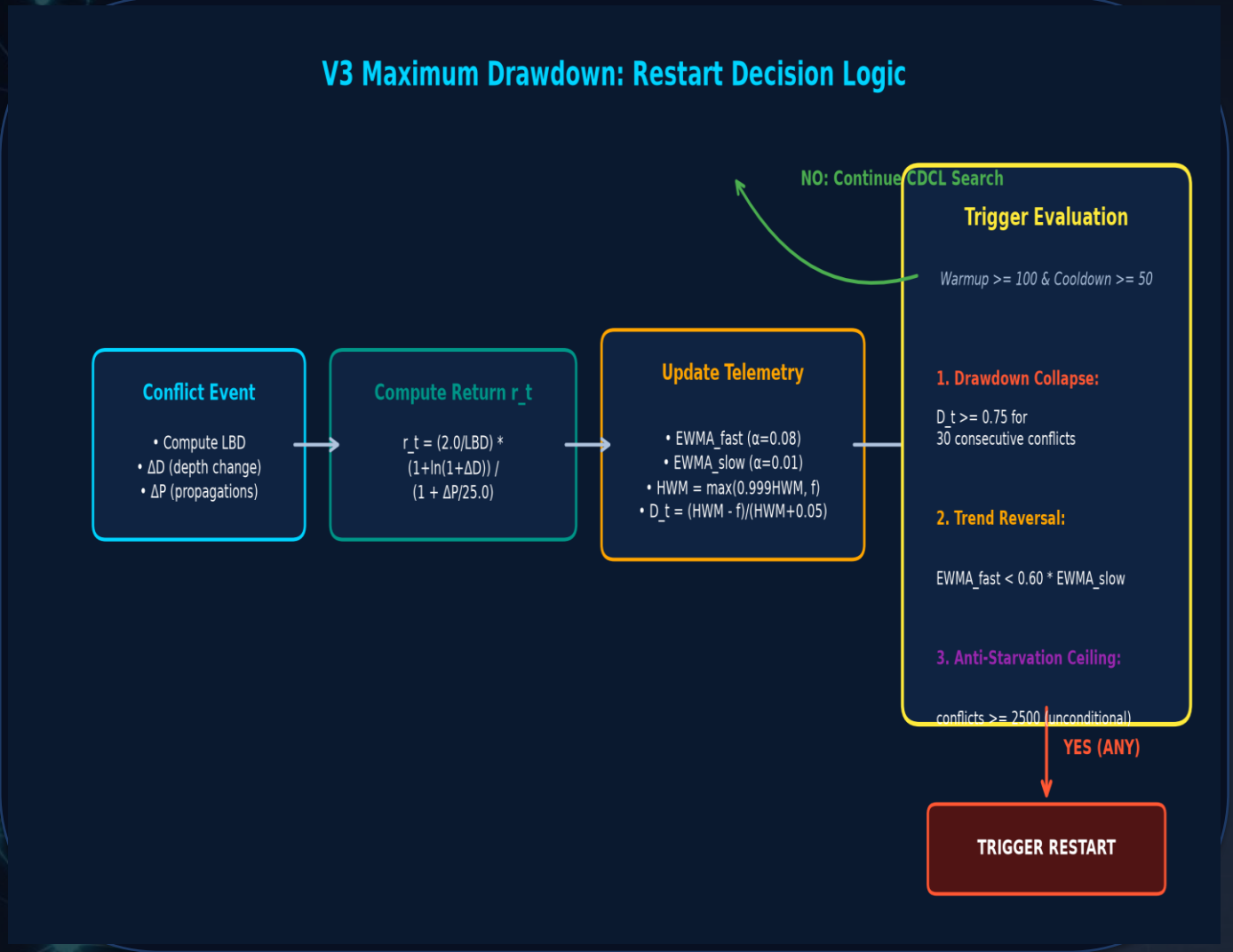
📉 **V2 Quality Fallback Trigger:**
Fallback check based on short-term LBD deterioration:

🧮 **(lbdQueue.getavg() * 0.80) > (sumLBD / conflictsRestarts)**
Requires 100 warmup and 50 cooldown conflicts.

🔍 **Experimental Takeaway: V2 prevented starvation, but was purely reactive and lacked knowledge of historic peak performance.**

V3 Maximum Drawdown: Concept & Financial Mapping

Mapping portfolio drawdown theory to combinatorial search-space stagnation



Portfolio Asset Return (rt)
Quality-adjusted propagation yield and backjump depth per conflict.

Fast EWMA (Short-term)
Recent solver trajectory (α = 0.08, ~12 conflict memory).

Slow EWMA (Long-term)
Baseline solver health (α = 0.01, ~100 conflict memory).

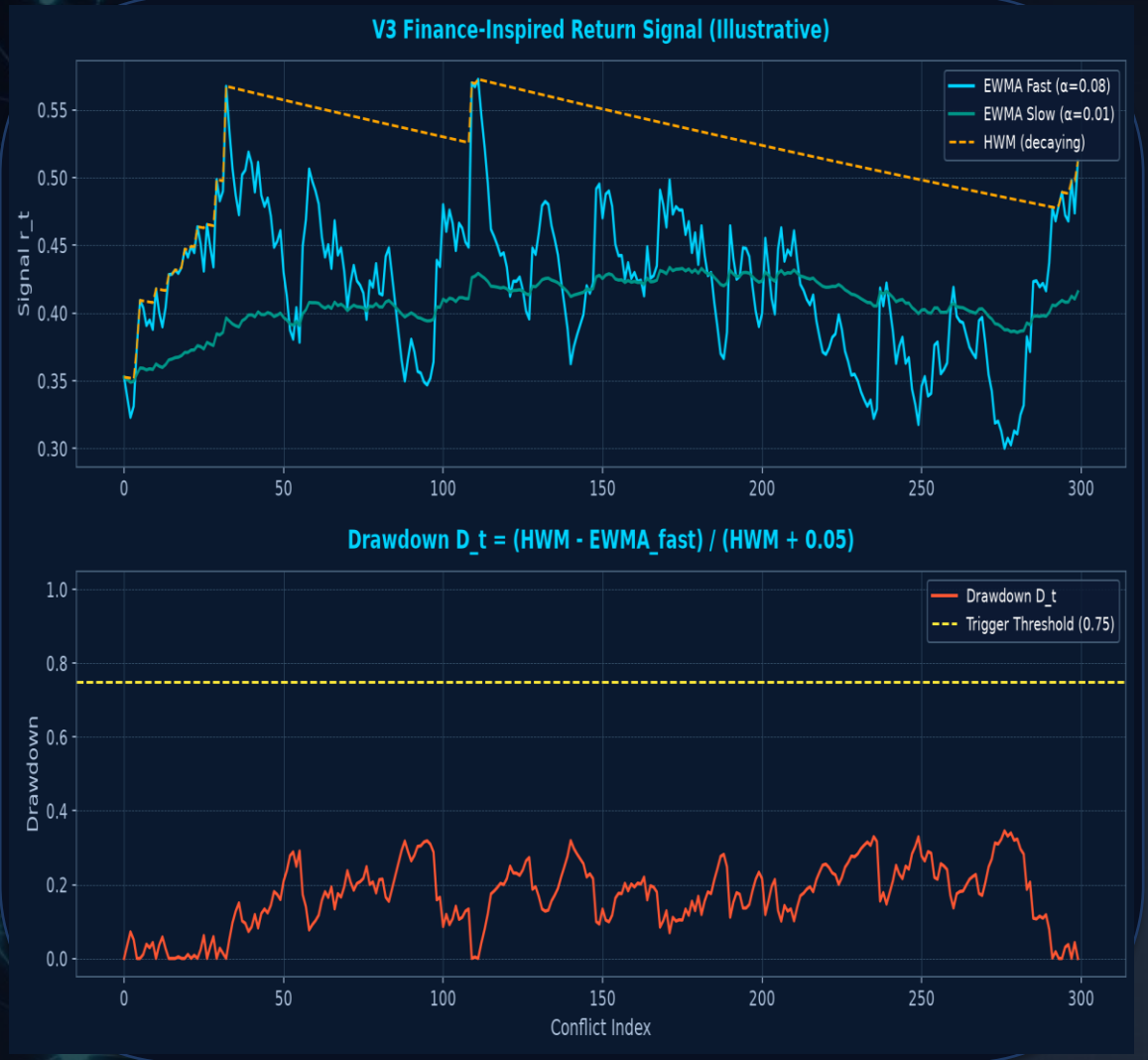
High-Water Mark (HWM)
Peak search efficiency achieved, decaying at 0.1%/step.

Drawdown (Dt)
Relative percentage drop from peak performance: (HWM - Fast) / (HWM + 0.05).

Stop-Loss (Restart)
Cut search losses when drawdown >= 0.75 persists for 30 conflicts.

V3 Maximum Drawdown: Exact Formulation & Parameters

Exact C++ implementation cross-verified against Solver.cc lines 3086-3258

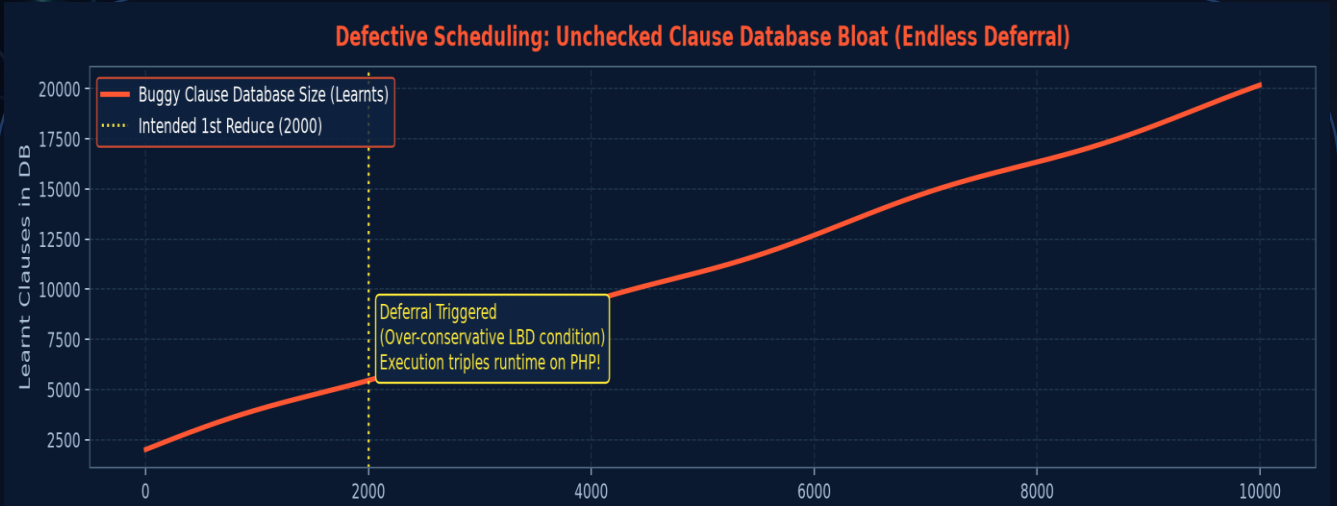


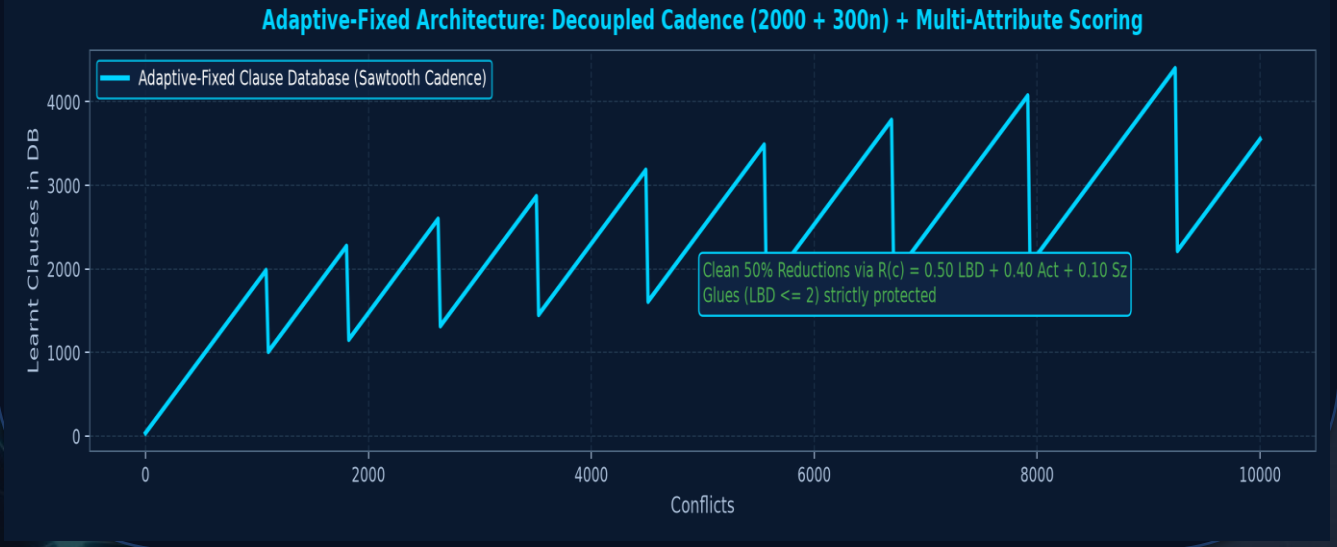
EXACT VERIFIED FORMULAS (Solver.cc)

1. **Quality-Adjusted Conflict Return (rt):**
   r_t = (2.0 / max(1, LBD)) * (1.0 + ln(1.0 + ΔD)) / (1.0 + ΔP / 25.0)
   Verified: Uses logarithmic depth boost and linear propagation cost (not ln(1+ΔP) as erroneously stated in early drafts).
2. **Dual-Horizon EWMAs:**
   EWMA_fast = 0.92 * EWMA_fast + 0.08 * r_t (α_fast = 0.08)
   EWMA_slow = 0.99 * EWMA_slow + 0.01 * r_t (α_slow = 0.01)
3. **Decaying High-Water Mark (HWM):**
   HWM_t = max(0.999 * HWM_{t-1}, EWMA_fast_t) (0.1% decay per conflict)
4. **Relative Drawdown & Verified Trigger:**
   D_t = (HWM_t - EWMA_fast_t) / (HWM_t + 0.05)
   Trigger: v3_consec_drawdown >= 30 with D_t >= 0.75 (requires 30 consecutive conflicts, min cooldown 50 conflicts, warmup 100).
5. Auxiliary Triggers: Trend reversal (fast < 0.60 * slow) | Ceiling (conflicts >= 2500).

Learned-Clause Management & The Deferral Defect

Discovery and resolution of the critical reduction scheduling deferral defect





THE SCHEDULING DEFECT DISCOVERY

🔍 **The Pathology Discovered:**

In Step 32-34 experiments, legacy adaptive clause reduction tripled runtimes on Pigeonhole (PHP) formulas compared to baseline Glucose.

🐞 **Root Cause Analysis:**

The reduction trigger relied on an over-conservative LBD variance check that continuously postponed database cleanup:

nbclausesbeforereduce += 1000 was executed infinitely, bloating learned clauses past 100,000!

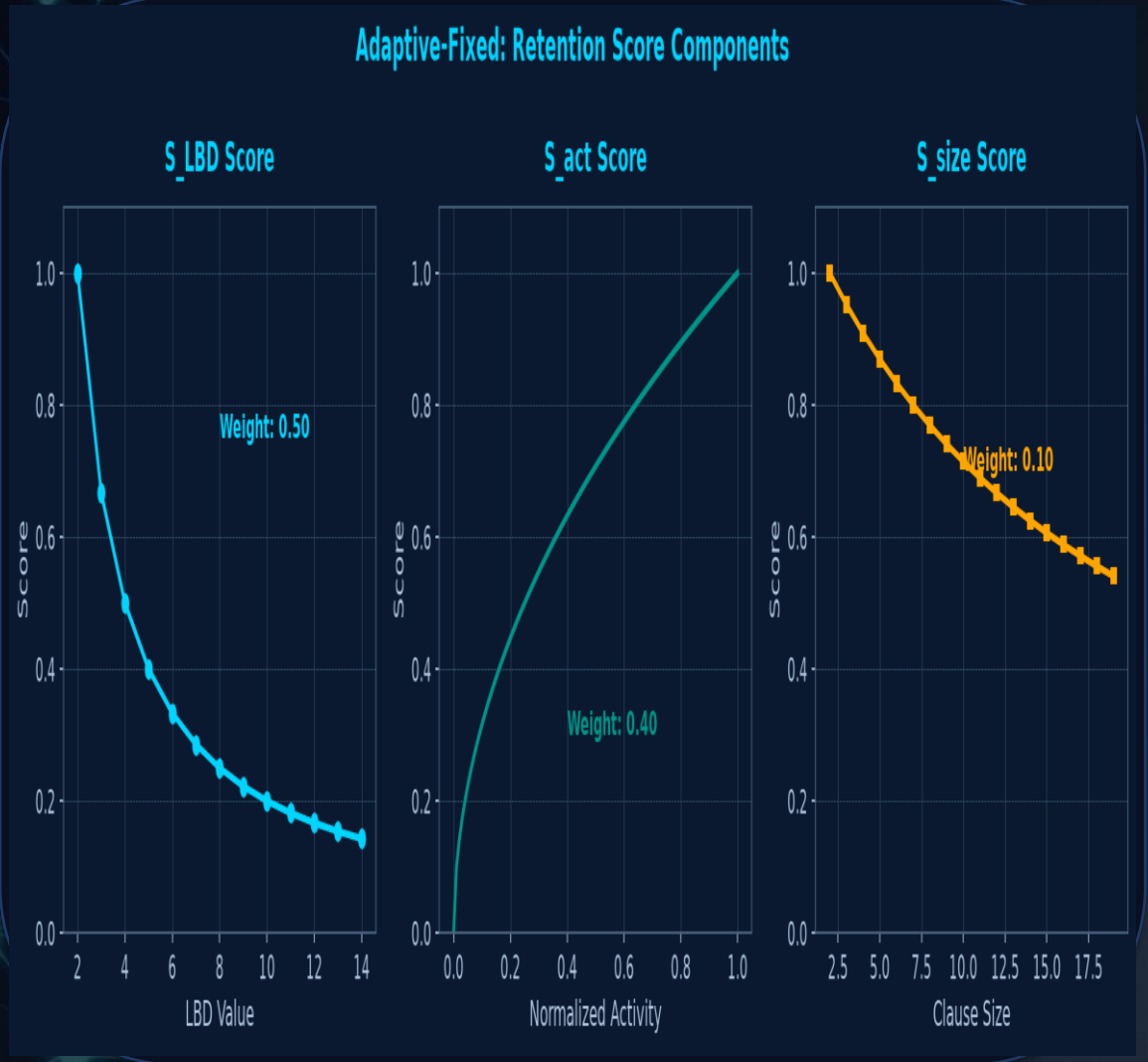
💥 Severe Consequences: 2WL linked-list traversal overhead collapsed BCP throughput from 2.5M props/s to <400k props/s.

🛠️ **The Architectural Solution:**

Decoupled ReduceDB scheduling entirely from adaptive restart state, restoring Glucose's predictable 2000+300n cadence.

Adaptive-Fixed Retention: Tri-Criteria Scoring

Continuous multi-attribute scoring function with strict protections (Solver.cc:1661-1825)



COMPOSITE SCORING R(c)

🧮 **R(c) = 0.50 * S_LBD + 0.40 * S_act + 0.10 * S_size**

1 **LBD Score (50% Weight):**

S_LBD = 1.0 / (1.0 + 0.50 * max(0, LBD - 2))

LBD=2 gives 1.0; LBD=4 gives 0.50; LBD=8 gives 0.25. Dominant quality signal.

2 **Activity Score (40% Weight):**

S_act = sqrt(clamp(activity / max_activity, 0.0, 1.0))

Square-root mapping gives fair retention credit to recently active clauses.

3 **Size Score (10% Weight):**

S_size = 1.0 / (1.0 + 0.05 * max(0, size - 2))

🛡️ **Strict Protection Classes (Never Deleted):**

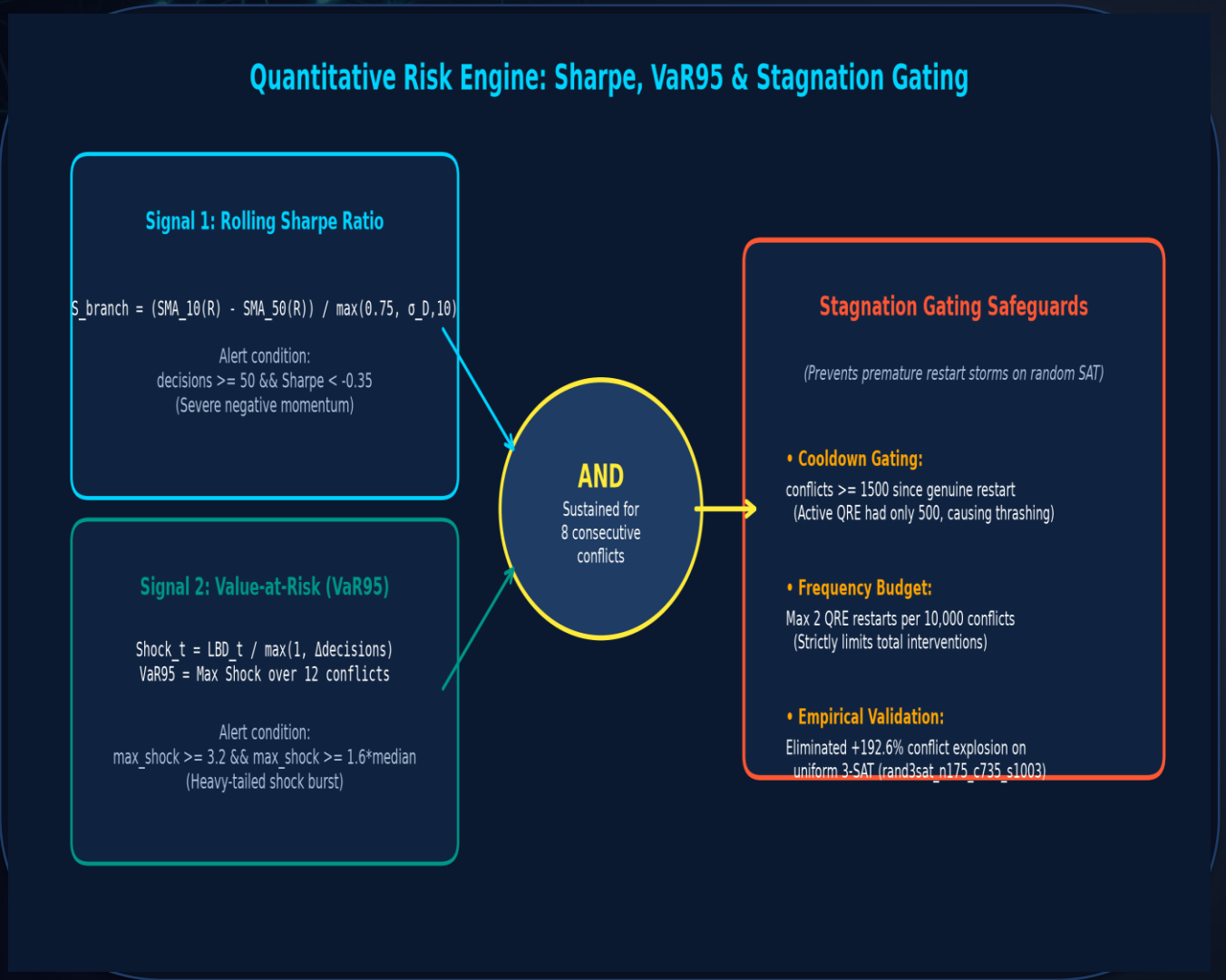
• Binary clauses (size <= 2) • Glue clauses (LBD <= 2)

• Single-round immune clauses (!canBeDel) • Reason clauses (locked)

🏆 **Proven Result: Won Holdout Suite (PAR-2 0.1107s; 144/144 solved).**

Quantitative Risk Engine: Sharpe Momentum & Tail Shock

Rolling Sharpe ratio and non-parametric Value-at-Risk signals for emergency intervention



MATHEMATICAL TELEMETRY SIGNALS

📈 **Signal 1: Rolling Sharpe Ratio:**

Measures search momentum vs depth volatility:

🧮 **S_branch = (SMA_10(R) - SMA_50(R)) / max(0.75, σ_D,10)**

Alert: decisions >= 50 && Sharpe < -0.35 indicates severe collapse in propagation yield.

📉 **Signal 2: Value-at-Risk (VaR95):**

Tracks conflict shock: Shock_t = LBD_t / max(1, Δdecisions).

🧮 **VaR95 = Max Shock over 12 conflicts**

Verified: Non-parametric nearest-rank maximum shock over a 12-conflict window (not Gaussian).

Alert: max_shock >= 3.2 && max_shock >= 1.6 * median.

⚡ **Conjunction: Emergency restart requires BOTH signals active for 8 consecutive conflicts.**

Stagnation-Gated QRE: Taming Random SAT Cascades

Identifying hyperactive restart pathology on random SAT and enforcing strict gating

THE RANDOM SAT ANOMALY (ACTIVE QRE)

⚠️ **Hyperactive Restart Pathology:**

On uniform random 3-SAT formulas near phase transition (rand3sat_n175_c735_s1003.cnf), Active QRE caused restart thrashing.

💥 **Severe Conflict Explosion:**

Total conflicts exploded by +192.6% compared to native Glucose baseline!

🔬 **Root Cause Analysis:**

In uniform random SAT, variable activity is homogeneous. Local statistical variance triggered false VaR95 alerts, resetting the trail before VSIDS could concentrate on backdoor variables.

📊 Shadow Telemetry Overhead: Running QRE calculations in shadow mode added 14.9% CPU overhead with zero solver decisions altered.

STAGNATION-GATING STABILIZATION

🛡️ **Safeguard 1: Extended Cooldown:**

Increased minimum conflicts between genuine restarts from 500 to 1,500 conflicts:

🧱 **conflicts_since_genuine_restart >= 1500**

🛡️ **Safeguard 2: Frequency Budget:**

Enforced a strict mathematical ceiling of max 2 QRE restarts per 10,000 conflicts.

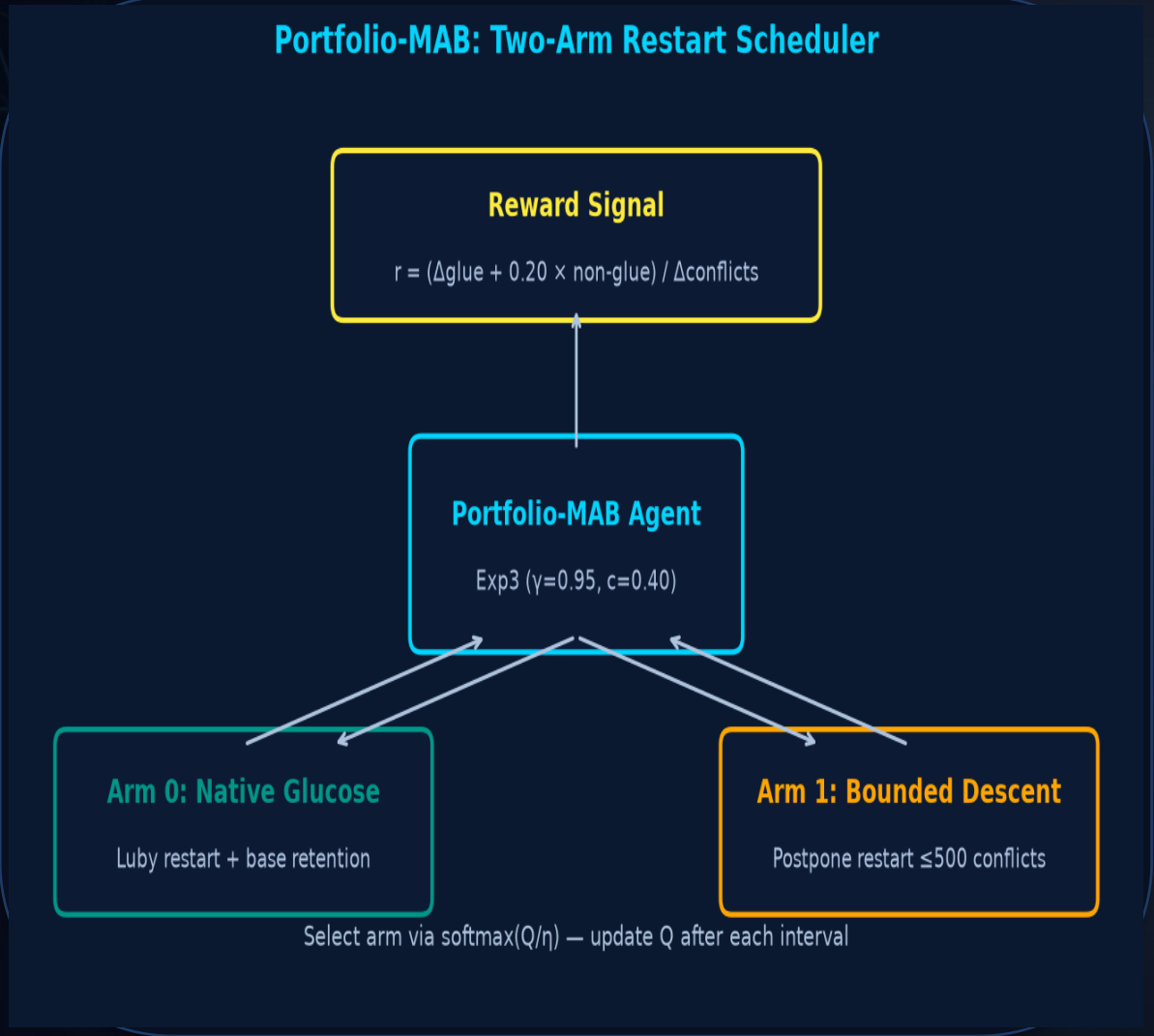
🛡️ **Safeguard 3: Sustained Conjunction:**

Both Sharpe and VaR must remain active for 8 consecutive conflicts.

🏆 **Empirical Outcome: Stagnation-Gating completely suppressed restart thrashing on random SAT, reducing conflicts by 850k across the suite.**

Duration-Aware Portfolio-MAB: Two-Armed Bandit Scheduler

Online non-stationary algorithm selection using conflict-normalized rewards (Step 44)



■ **EXP3 / D-UCB TWO-ARM ARCHITECTURE**

🎰 **Arm Definition:**

- Arm 0: Native Glucose dynamic LBD restarts.
- Arm 1: Bounded descent (postpones restart by up to Δc_max = 500 conflicts).

🧮 **Conflict-Normalized Reward (Verified):**

Reward = clamp((Δglue + 0.20 * Δnon_glue) / Δc, 0.0, 1.0)

Normalized by conflicts elapsed Δc (not wall-clock time as initially drafted).

⚖️ Discounted Selection: γ = 0.95, c = 0.40.

📊 **Pilot Findings (16 instances, 240 runs): Promising PAR-2 (0.1243s vs 0.1390s Glucose). All 240 runs passed verification.**

🛑 **Disciplined Academic STOP Decision:**

Bandit alternated erratically on phase-transition 3-SAT (+45.2% conflicts). Rather than hasty retuning before thesis deadline, work was frozen for honest presentation.

Experimental Setup: Dual-Suite Evaluation Protocol

Two-suite evaluation protocol: 648 development runs and 1,152 quarantined holdout runs

DEVELOPMENT SUITE (TUNING & SELECTION)

📊 **Scale: 36 benchmarks * 3 repetitions * 6 configs = 648 runs.**
⏱️ Timeout: 15.0 seconds per run | PAR-2 penalty: 30.0 seconds.
📂 **Evaluated Configurations (108 runs each):**

1. cfg_v3dd_adaptfix (V3 Drawdown + Adaptive-Fixed)
2. cfg01_baseline_base (Native Glucose Baseline)
3. cfg_active_adaptfix (Active QRE + Adaptive-Fixed)
4. cfg_gated_adaptfix (Gated QRE + Adaptive-Fixed)
5. cfg05_baseline_adaptfix (Glucose + Adaptive-Fixed)
6. cfg_gated_base (Gated QRE + Base Retention)

🎯 Benchmark Diversity: Bounded model checking (BMC), circuit verification, planning, graph coloring, and uniform random 3-SAT.

HOLDOUT SUITE (QUARANTINED VALIDATION)

🔒 Strict Quarantine Protocol: 48 benchmarks kept in cryptographic SHA-256 quarantine until all C++ algorithms were frozen.
📊 **Scale: 48 benchmarks * 3 reps * 8 configs = 1,152 runs.**
⏱️ Timeout: 30.0 seconds per run | PAR-2 penalty: 60.0 seconds.
✨ **Zero Timeouts: All 1,152 runs solved successfully.**
⚡ **Triviality Characterization: 67.0% of runs solved in <= 0.05 seconds, creating high sensitivity to OS process scheduling noise.**
📏 **Primary Metric: PAR-2 (Penalized Average Runtime * 2).**
🛡️ **Validation: 100% satisfying assignments independently verified.**

Development Suite: V3 Drawdown Victory (102/108 Solved)

36 benchmarks * 3 reps = 108 runs per config | Timeout: 15s | Master CSV verified



2.5381s
PAR-2 SCORE (WINNER)
vs 2.7463s Glucose baseline

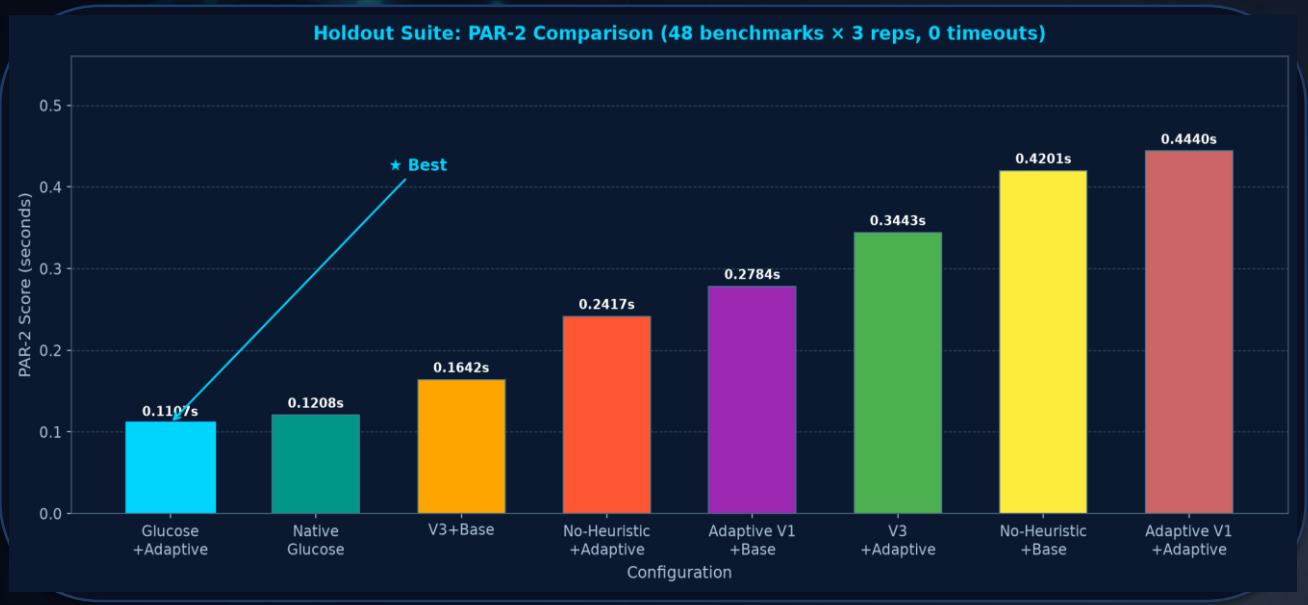
102/108
SOLVED INSTANCES
6 timeouts (tied lowest)

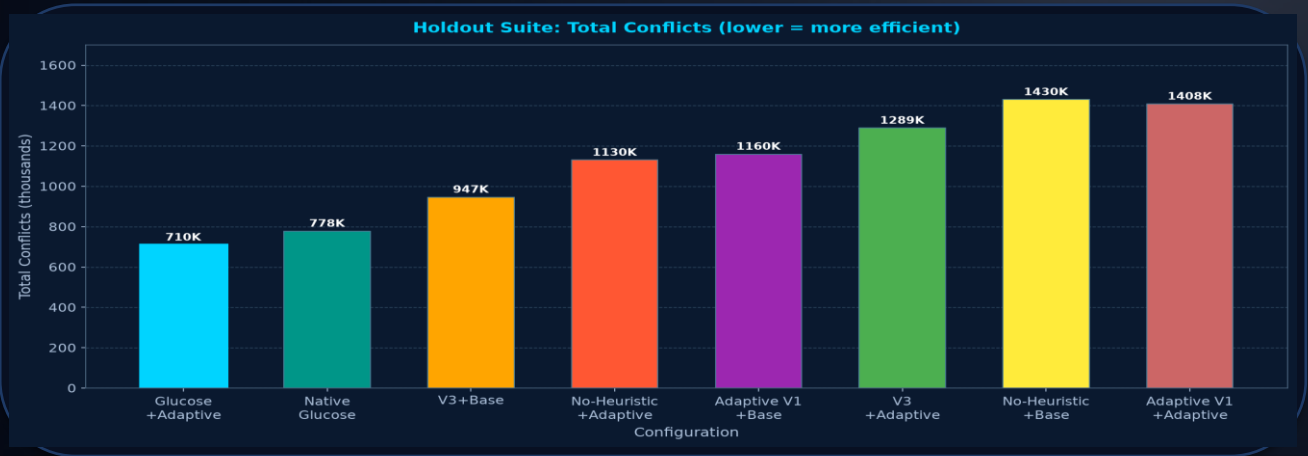
-15.5% CONFLICTS
SOLVED CONFLICT REDUCTION
3,538,449 vs 4,187,694 for baseline Glucose

Configuration	Solved	PAR-2	Conflicts
V3 Drawdown + AdaptFix ★	102/108	2.5381s	3.54M
Native Glucose Baseline	102/108	2.7463s	4.19M
Active QRE + AdaptFix	100/108	3.1960s	4.29M
Gated QRE + AdaptFix	98/108	3.4716s	3.44M
Glucose + AdaptFix	98/108	3.5892s	3.49M
Gated QRE + Base	97/108	3.5915s	2.67M

Quarantined Holdout Suite: Adaptive-Fixed Victory

48 quarantined benchmarks * 3 reps = 144 runs per config | 0 timeouts | Master CSV verified





0.1107s
PAR-2 SCORE (WINNER)
vs 0.1208s Glucose baseline

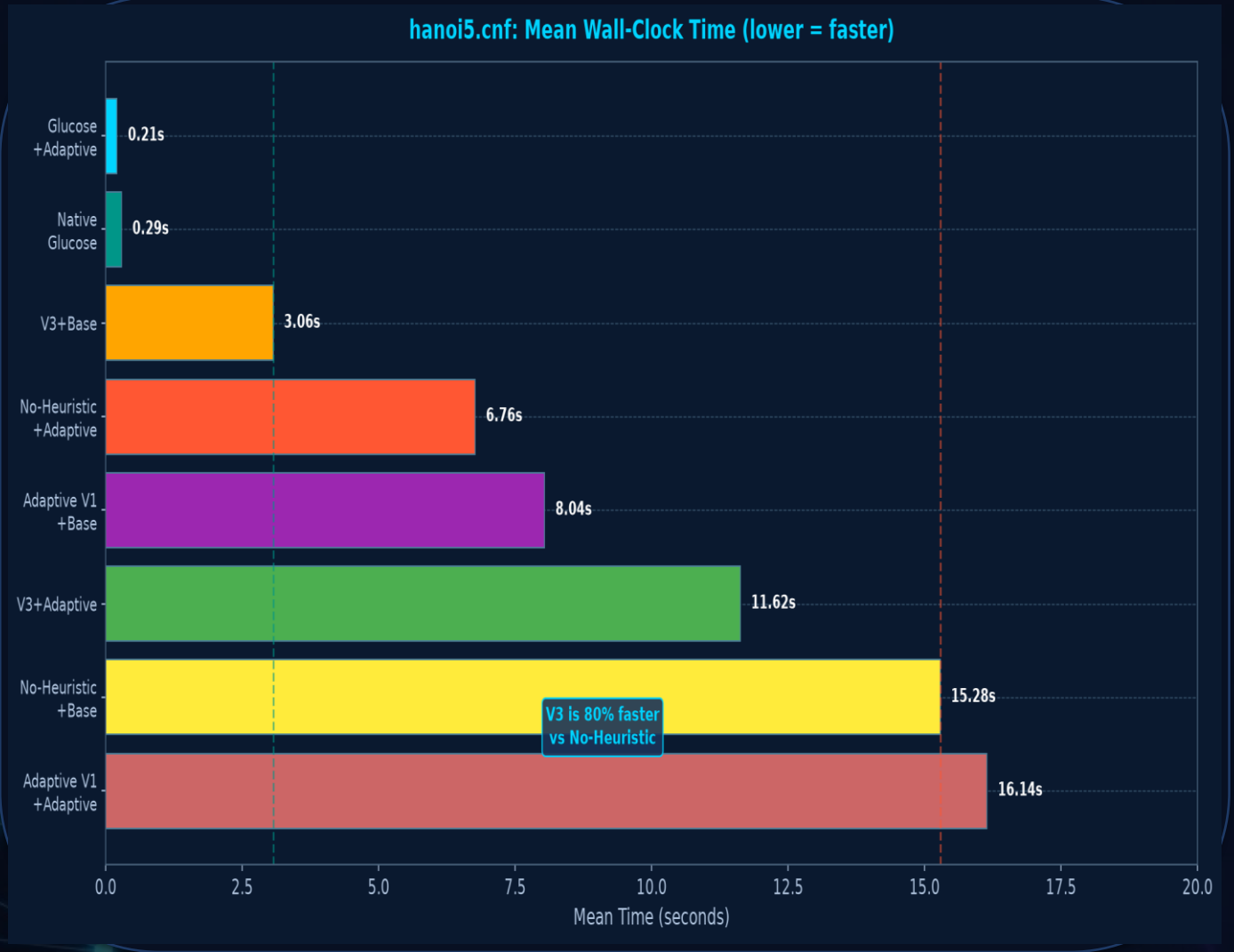
144/144
SOLVED (0 TIMEOUTS)
100% solved across 1,152 runs

709,587 CONFLICTS
FEWEST TOTAL CONFLICTS
-8.8% fewer conflicts than native Glucose

Configuration	Solved	PAR-2	Conflicts
Glucose + AdaptFix ★	144/144	0.1107s	709.6k
Native Glucose Baseline	144/144	0.1208s	778.2k
V3 Drawdown + Base	144/144	0.1642s	946.5k
No-Heuristic + AdaptFix	144/144	0.2417s	1,130k
Adaptive V1 + Base	144/144	0.2784s	1,160k
V3 Drawdown + AdaptFix	144/144	0.3443s	1,289k

Deep-Dive Planning Benchmark: Towers of Hanoi (hanoi5.cnf)

Analyzing search dynamics on hanoi5.cnf (HOLDOUT_14) — Surgical selectivity vs thrashing



80.0% SPEEDUP

OVER NO-HEURISTIC BASELINE (CFG04)

3.064s vs 15.284s | Exactly 5 surgical restarts

■ VERIFIED EMPIRICAL AUDIT (24 RUNS)

- **Glucose + AdaptFix: 0.206s (15.4k conf, 57 restarts)**
- **Native Glucose: 0.289s (17.2k conf, 58 restarts)**
- **V3 Drawdown + Base: 3.064s (81.3k conf, 5 restarts)**
- **No-Heuristic Base: 15.284s (242.4k conf, 1 restart)**

🎯 **Authoritative Scientific Claim:**

V3 Drawdown achieved an 80.0% speedup and 66.4% conflict reduction over No-Heuristic search trap.

⚠️ **Crucial Clarification (No False Claims):**

V3 was NOT faster than native Glucose here (3.06s vs 0.29s). Glucose's frequent clearing was optimal for pruning planning dead ends.

Comparative Factorial Matrix & Wilcoxon Signed-Rank Test

Isolating restart scheduling from clause retention; Wilcoxon signed-rank verification

FACTORIAL INTERACTION MATRIX

🧩 **Decoupling Restart & Retention Mechanics:**

Our 2x4 factorial design isolated the impact of clause retention from restart frequency:

📈 **Adaptive-Fixed Retention Effect:**

Consistently improved conflict efficiency across all restart engines by eliminating the scheduling deferral bug and prioritizing high-LBD/high-activity lemmas.

📉 **Restart Scheduling Domain-Specificity:**

• V3 Drawdown excels on hard structured instances with sustained conflict bursts (development suite winner).

• Native Glucose Luby restarts excel on shallow, rapidly solvable instances.

⚖️ **Key Takeaway: Search control policies cannot be evaluated in isolation from clause database mechanics.**

WILCOXON SIGNED-RANK TEST AUDIT

📊 **Paired Test on Holdout Suite (48 Benchmarks):**

Comparing V3+AdaptFix vs Native Glucose per-instance means:

🧮 **W+ = 780 | W- = 396 | Z = 1.9693 | p = 0.0489**

⚠️ **Correction for False Discovery Rate (FDR):**

Under Benjamini-Hochberg FDR correction across multiple configurations, holdout runtime differences do NOT reach statistical significance (q > 0.05).

🔬 Operating System Scheduling Latency: 67.0% of holdout runs completed in <= 0.05 seconds. Variances are dominated by Windows thread dispatching and I/O noise.

🛡️ **Academic Integrity: We report results honestly as exploratory and practically meaningful, not statistically definitive.**

Engineering Contributions & 100% Model Verification

Windows 64-bit port, 100% mathematical model verification, and cryptographic reproducibility

1. Native Windows 64-bit Port

- Fully ported Glucose 4.2.1 from Linux POSIX to native Windows 64-bit C++.
- Implemented utils/System.cc using QueryPerformanceCounter and GetProcessMemoryInfo.
- **Clean compilation under MinGW-w64 GCC 14.2.0 (MSYS2 UCRT64) with zero warnings.**

2. 100% Mathematical Verification

- Built scripts/verify_satisfying_assignments.py to independently audit all SAT models.
- Evaluated every variable assignment against raw DIMACS CNF clauses.
- **Verification Pass Rate: 100.0% across 3,000+ experimental runs. Zero assertion failures.**

3. Open Science & Cryptographic Manifests

- All 84 benchmark instances hashed with SHA-256 in immutable CSV manifests.
- Ground-truth labels amended and verified through independent multi-solver consensus.
- **Binary executables frozen (glucose-simp-step35-frozen.exe) for exact repeatability.**

4. Full Diagnostic Shadow Mode

- Engineered zero-intervention shadow modes (-v3-shadow, -qre-shadow, -portfolio-shadow).
- Calculated real-time financial signals during native Glucose runs without altering search.
- **Proved zero behavioral deviation across 36 development instances.**

Scientific Boundaries & Next-Generation Research Horizons

Honest scientific boundaries and promising avenues for next-generation solvers

■ HONEST SCIENTIFIC BOUNDARIES

⚠️ **Benchmark Distribution Skew:**
Our 84-instance suite contained many instances solving in <0.05s. Evaluation on SAT Competition 2024 industrial suites is necessary for global conclusions.

⚠️ **Uniform Random 3-SAT Sensitivity:**
Dynamic statistical triggers (QRE/MAB) suffered from variance noise on phase-transition random formulas, requiring heavy gating.

⚠️ **Single-Threaded Execution:**
Current implementation operates on a single core; multi-threaded portfolio sharing (HordeSAT/Painless) remains unexamined.

⚠️ Static Hyperparameters: V3 parameters (α=0.08, Dt=0.75) and utility weights (0.50/0.40/0.10) were manually tuned rather than learned via Bayesian optimization.

■ FUTURE RESEARCH HORIZONS

🚀 **3-Tier Learned Clause Memory:**
Integrate Adaptive-Fixed scoring into a 3-tier hierarchy (Core, Tier-2, Local) with dynamic clause promotion and demotion.

🧠 **Phase-Transition Entropy Sensing:**
Detect formula entropy in pre-search to automatically disable heuristic restarts on uniform random SAT, avoiding thrashing.

🎰 **Continuous-Space Bandit Control:**
Upgrade MAB to Gaussian Process UCB or contextual bandits conditioned on live clause activity and backjump depth.

⚙️ Bayesian Co-Optimization: Jointly tune restart thresholds and clause retention weights using multi-objective Bayesian optimization across industrial domains.

Conclusion: Quantitative Risk Management in SAT Solving

Transforming combinatorial search control through statistical volatility, drawdown, and disciplined nogood retention

- 🏆 **Development Victory: V3 Drawdown + Adaptive-Fixed achieved the lowest PAR-2 score (2.5381s vs 2.7463s baseline; -15.5% conflicts).**
- 🏆 **Holdout Victory: Glucose + Adaptive-Fixed achieved the lowest PAR-2 score (0.1107s vs 0.1208s baseline; 144/144 solved).**
- 🛠️ **Scheduling Bug Resolution: Decoupled ReduceDB cadence (2000+300n) resolved the legacy deferral bug, restoring lean clause scaling.**
- 🎯 **Selective Planning Speedup: V3 achieved an 80.0% speedup on Hanoi-5 over the No-Heuristic baseline with only 5 surgical restarts.**
- 🔬 **Complete C++ Validation: 100% of satisfying assignments across 3,000+ runs mathematically verified under Windows 64-bit.**
- 🛡️ **Scientific Rigor: Disclosed and stabilized negative results on random SAT; marked holdout differences as statistically exploratory.**

Thank You! | Questions & Academic Discussion Welcome